# CSM GLEAMING PART 2 Meta

COMPUTER SCIENCE MENTORS 61A

November 9, 2026 – November 13, 2026

## 1 More Gleam!

These questions are adopted from past CS 61A exams related to linked lists, trees, and/or Scheme. You can find a list of useful builtin functions in Gleam here:
https://gleam-stdlib.hexdocs.pm/gleam/list.html
https://gleam-stdlib.hexdocs.pm/gleam/int.html
https://gleam-stdlib.hexdocs.pm/gleam/result.html

1. (Adapted from CS 61A Fall 2024 Final Q7 (A Pair of Schemes) points) Implement `all_pairs`, which takes a two-argument function `f` and a list `s`. It returns `True` if `f(x, y)` returns `True` for every pair of adjacent elements `x` and `y` in `s`. The function `inc` returns whether `x + 1` equals `y`.

```
pub fn inc(x: Int, y: Int) -> Bool {
  x + 1 == y
}

pub fn all_pairs(f: fn(a, a) -> Bool, s: List(a)) -> Bool {
  case s {
    [x, y, ..rest] -> _____________ && all_pairs(f, _____________)
    _ -> True
  }
}
assert all_pairs(inc, [3, 4, 5, 6, 7, 8]) == True
assert all_pairs(inc, [3, 4, 5, 8, 7, 8]) == False
assert all_pairs(inc, [3, 4, 6, 7]) == False
assert all_pairs(inc, [3]) == True
assert all_pairs(inc, []) == True
```

```
pub fn all_pairs(f: fn(a, a) -> Bool, s: List(a)) -> Bool {
  case s {
    [x, y, ..rest] -> f(x, y) && all_pairs(f, [y, ..rest])
    _ -> True
  }
}
```

Created by Ana Abell, Terry Hsu. feedback form!
Provided by CSM 61A contributors under CC BY 4.0/3-clause BSD.

2. Implement `show_pairs`, which takes a list `s` and returns a list of all pairs of adjacent elements in `s`, in order. Each pair is a tuple.

```
pub fn show_pairs(s: List(a)) -> List(#(a, a)) {
  case s {
    [x, y, ..rest] -> [__________, ..show_pairs(________________)]
    _ -> []
  }
}
assert show_pairs([3, 5, 7, 9, 11, 13])
  == [#(3, 5), #(5, 7), #(7, 9), #(9, 11), #(11, 13)]
assert show_pairs([1, 2]) == [#(1, 2)]
assert show_pairs([1]) == []
```

```
pub fn show_pairs(s: List(a)) -> List(#(a, a)) {
  case s {
    [x, y, ..rest] -> [#(x, y), ..show_pairs([y, ..rest])]
    _ -> []
  }
}
```

This has the same structure as `all_pairs`, but instead of combining each pair's result with `&&`, it collects the pairs themselves into a list. As before, the recursive call starts at `y` because adjacent pairs overlap. A list with $n \geq 1$ elements has $n - 1$ adjacent pairs, so the result is always one element shorter than `s`.

3. Gleam cannot build an expression and evaluate it later the way Scheme's `eval` can, but it can build a list of *delayed calls*: functions that take no arguments and make a call only when they are called. Implement `all_pairs_calls`, which takes a two-argument function `f` and a list `s`. It returns one delayed call for each pair of adjacent elements `x` and `y` in `s`, which calls `f(x, y)` when it is called. Then implement `run_all`, which calls each delayed call in a list and returns whether all of them returned `True`. You may use `show_pairs`.

```
pub fn all_pairs_calls(f: fn(a, a) -> Bool, s: List(a)) -> List(fn() ->
    Bool) {
  list.map(show_pairs(s), fn(pair) { ____________________________________ })
}

pub fn run_all(calls: List(fn() -> Bool)) -> Bool {
  list.all(calls, _________________________)
}
assert list.length(all_pairs_calls(inc, [3, 4, 5, 6, 7, 8])) == 5
assert run_all(all_pairs_calls(inc, [3, 4, 5, 6, 7, 8])) == True
assert run_all(all_pairs_calls(inc, [3, 4, 5, 8, 7, 8])) == False
assert run_all([]) == True
```

```
pub fn all_pairs_calls(f: fn(a, a) -> Bool, s: List(a)) -> List(fn() ->
    Bool) {
  list.map(show_pairs(s), fn(pair) { fn() { f(pair.0, pair.1) } })
}

pub fn run_all(calls: List(fn() -> Bool)) -> Bool {
  list.all(calls, fn(call) { call() })
}
```

4. (Adapted from CS 61A Fall 2023 Final Q6 (How to Get Promoted) points) Implement `promote`, which takes a function `f` and a list `s`. It returns a list containing all of the elements of `s`, with the elements for which `f` returns `True` at the front. Otherwise, the elements stay in the same order as in `s`.

```
pub fn promote(f: fn(a) -> Bool, s: List(a)) -> List(a) {
  ___________(_________________, list.filter(s, _____________________))
}
assert promote(fn(x) { x % 2 == 0 }, [1, 2, 3, 4, 5, 6, 7])
  == [2, 4, 6, 1, 3, 5, 7]
assert promote(fn(x) { x % 2 == 1 }, [1, 2, 3, 4, 5, 6, 7])
  == [1, 3, 5, 7, 2, 4, 6]
assert promote(fn(x) { x > 100 }, [1, 2, 3]) == [1, 2, 3]
```

```
pub fn promote(f: fn(a) -> Bool, s: List(a)) -> List(a) {
  list.append(list.filter(s, f), list.filter(s, fn(x) { !f(x) }))
}
```

5. Implement `bigger_first`, which takes a non-empty list `s`. It returns a list with all of the elements of `s` that are larger than the first element at the front, followed by all of the elements that are smaller than or equal to the first element. Each group keeps its original order. You may use `promote` from the previous question and the functions below. **You may not write** `fn`, `case`, `[`, or `]` in the blank.

```
pub fn curry(f: fn(a, b) -> c) -> fn(a) -> fn(b) -> c {
  fn(x) { fn(y) { f(x, y) } }
}

pub fn curry_call(f: fn(x, y) -> r) -> fn(fn(z) -> x, fn(y) -> z) -> fn(y)
    -> r {
  fn(x, g) { fn(y) { f(x(g(y)), y) } }
}

pub fn less_than(x: Int, y: Int) -> Bool {
  x < y
}

pub fn first(s: List(Int)) -> Int {
  let assert [x, ..] = s
  x
}

pub fn bigger_first(s: List(Int)) -> List(Int) {
  ____________________________________________(s)
}
assert bigger_first([3, 1, 4, 1, 5, 9, 2, 6]) == [4, 5, 9, 6, 3, 1, 1, 2]
assert bigger_first([5, 1, 2]) == [5, 1, 2]
assert bigger_first([1, 2, 3]) == [2, 3, 1]
```

```
pub fn bigger_first(s: List(Int)) -> List(Int) {
  curry_call(promote)(curry(less_than), first)(s)
}
```

6. (Adapted from CS 61A Fall 2022 Final Q4b (Advent of Code) points) Implement `find_pair`, which takes a list of integers `s` and returns the two elements of `s` that sum to 2020, in the order that they appear in `s`. Assume that exactly one pair of elements sums to 2020. You may call **`list`**`.contains(s, x)`, which returns whether `s` contains `x`.

```
pub fn find_pair(s: List(Int)) -> List(Int) {
  case s {
    [] -> []
    [x, ..rest] ->
      case ____________________________________ {
        True -> ___________________
        False -> ___________________
      }
  }
}
assert find_pair([979, 1721, 366, 299, 675, 1456]) == [1721, 299]
assert find_pair([2000, 20]) == [2000, 20]
assert find_pair([1, 1010, 3, 1010]) == [1010, 1010]
```

```
pub fn find_pair(s: List(Int)) -> List(Int) {
  case s {
    [] -> []
    [x, ..rest] ->
      case list.contains(rest, 2020 - x) {
        True -> [x, 2020 - x]
        False -> find_pair(rest)
      }
  }
}
```

7. (Adapted from CS 61A Fall 2014 Midterm 2 Q3b (This One Goes to Eleven) points) Implement `no_eleven`, which takes a non-negative integer `n`. It returns a list of all distinct lists of length `n` that contain only `1`s and `6`s, in which two `1`s never appear next to each other.

```
pub fn no_eleven(n: Int) -> List(List(Int)) {
  case n {
    0 -> _________
    1 -> [[6], [1]]
    _ ->
      list.append(
        list.map(_______________, fn(s) { [6, ..s] }),
        list.map(_______________, fn(s) { ________________ }),
      )
  }
}
assert no_eleven(0) == [[]]
assert no_eleven(2) == [[6, 6], [6, 1], [1, 6]]
assert no_eleven(3)
  == [[6, 6, 6], [6, 6, 1], [6, 1, 6], [1, 6, 6], [1, 6, 1]]
assert no_eleven(4)
  == [[6, 6, 6, 6], [6, 6, 6, 1], [6, 6, 1, 6], [6, 1, 6, 6],
      [6, 1, 6, 1], [1, 6, 6, 6], [1, 6, 6, 1], [1, 6, 1, 6]]
```

```
pub fn no_eleven(n: Int) -> List(List(Int)) {
  case n {
    0 -> [[]]
    1 -> [[6], [1]]
    _ ->
      list.append(
        list.map(no_eleven(n - 1), fn(s) { [6, ..s] }),
        list.map(no_eleven(n - 2), fn(s) { [1, 6, ..s] }),
      )
  }
}
```

8. (Adapted from CS 61A Spring 2018 Midterm 2 Q4b (Sequences) points) Implement `combo`, which takes two lists of digits `a` and `b`. It returns the smallest list of digits that contains all of the digits of `a` in order, as well as all of the digits of `b` in order. A list is smaller than another if it is shorter, or if they have the same length and it comes first when compared digit by digit. You may use `smaller`, which returns the smaller of two lists.

```
pub fn smaller(s: List(Int), t: List(Int)) -> List(Int) {
  // Implementation omitted
}

pub fn combo(a: List(Int), b: List(Int)) -> List(Int) {
  case a, b {
    [], _ -> ____
    _, [] -> a
    [x, ..ar], [y, ..br] if x == y -> _________________________
    [x, ..ar], [y, ..br] ->
      smaller(_____________________, _____________________)
  }
}
assert combo([5, 3, 1], [4, 3, 2]) == [4, 5, 3, 1, 2]
assert combo([5, 3, 1], [4, 3, 2, 1]) == [4, 5, 3, 2, 1]
assert combo([1, 2, 3, 4], [9, 1, 2, 3]) == [9, 1, 2, 3, 4]
assert combo([], [3, 2, 1]) == [3, 2, 1]
```

```
pub fn combo(a: List(Int), b: List(Int)) -> List(Int) {
  case a, b {
    [], _ -> b
    _, [] -> a
    [x, ..ar], [y, ..br] if x == y -> [x, ..combo(ar, br)]
    [x, ..ar], [y, ..br] ->
      smaller([x, ..combo(ar, b)], [y, ..combo(a, br)])
  }
}
```

1. Implement `simplify`, which takes in an `Expr expr` and returns an equivalent expression in which every `Binary` expression whose two sides both simplify to numbers is replaced by the single number it computes. Parts of `expr` that contain variables are kept. For example, `(x + (2 * 3))` simplifies to `(x + 6)`. Use the provided `calculate` function.

```
pub fn simplify(expr: Expr) -> Expr {
  case expr {
    Binary(operator, left, right) ->
      case _____________, _____________ {
        N(a), N(b) -> ____________________________
        l, r -> ____________________________
      }
    _ -> _____________
  }
}

assert simplify(Binary(Add, Var("x"), Binary(Mul, N(2), N(3))))
  == Binary(Add, Var("x"), N(6))
assert simplify(Binary(Mul, Binary(Sub, N(5), N(1)), N(2))) == N(8)
assert simplify(Binary(Add, Binary(Add, Var("x"), N(1)), Binary(Add, N(2),
   N(3))))
  == Binary(Add, Binary(Add, Var("x"), N(1)), N(5))
assert simplify(Var("x")) == Var("x")
```

```
pub fn simplify(expr: Expr) -> Expr {
  case expr {
    Binary(operator, left, right) ->
      case simplify(left), simplify(right) {
        N(a), N(b) -> N(calculate(operator, a, b))
        l, r -> Binary(operator, l, r)
      }
    _ -> expr
  }
}
```

2. Implement `substitute`, which takes in an `Expr` `expr`, a variable `name`, and an `Expr` `replacement`, and returns `expr` with every occurrence of the variable `name` replaced by `replacement`. All other parts of `expr` stay the same. For example, substituting `N(4)` for `"x"` in `(x + y)` gives `(4 + y)`.

```
pub fn substitute(expr: Expr, name: String, replacement: Expr) -> Expr {
  case expr {
    Var(v) if _____________ -> _____________
    Var(_) | N(_) -> _____________
    Binary(operator, left, right) ->
      ____________________________________________________________
  }
}

assert substitute(Binary(Add, Var("x"), Var("y")), "x", N(4))
  == Binary(Add, N(4), Var("y"))
assert substitute(Binary(Mul, Var("x"), Var("x")), "x", Binary(Sub,
   Var("a"), N(1)))
  == Binary(Mul, Binary(Sub, Var("a"), N(1)), Binary(Sub, Var("a"), N(1)))
assert substitute(Binary(Mul, N(2), Var("y")), "x", N(9))
  == Binary(Mul, N(2), Var("y"))
```

```
pub fn substitute(expr: Expr, name: String, replacement: Expr) -> Expr {
  case expr {
    Var(v) if v == name -> replacement
    Var(_) | N(_) -> expr
    Binary(operator, left, right) ->
      Binary(operator, substitute(left, name, replacement),
         substitute(right, name, replacement))
  }
}
```

3. Implement `curry`, which takes in a `UserFn f` and an integer `arg`, and returns a function that takes all but the first parameter of `f`, with `arg` bound to the first parameter. For example, currying `fn(x, y){ (y - x)}` with `1` gives `fn(y){ (y - 1)}`. If `f` has no parameters, return `Error(WrongArgumentCount(want: 0, got: 1))`. You may use `substitute` from the previous question.

```
pub fn curry(f: UserFn, arg: Int) -> Result(UserFn, EvalError) {
  case _____________ {
    [] -> Error(WrongArgumentCount(want: 0, got: 1))
    [first, ..rest] -> ____________________________________________
  }
}

assert curry(UserFn(["x", "y"], Binary(Sub, Var("y"), Var("x"))), 1)
  == Ok(UserFn(["y"], Binary(Sub, Var("y"), N(1))))
assert curry(UserFn(["c"], Binary(Add, N(10), Var("c"))), 1)
  == Ok(UserFn([], Binary(Add, N(10), N(1))))
assert curry(UserFn([], N(7)), 1) == Error(WrongArgumentCount(want: 0,
   got: 1))


pub fn curry(f: UserFn, arg: Int) -> Result(UserFn, EvalError) {
  case f.params {
    [] -> Error(WrongArgumentCount(want: 0, got: 1))
    [first, ..rest] -> Ok(UserFn(rest, substitute(f.body, first, N(arg))))
  }
}
```

4. Implement `compose`, which takes in two `UserFn`s `f` and `g`, and returns a function `h` such that applying `h` to some arguments gives the same result as applying `g` to those arguments and then applying `f` to that result. For example, composing `fn(x){ (x * x)}` with `fn(x, y){ (y - x)}` gives `fn(x, y){ ((y - x)* (y - x))}`. Since `f` receives exactly one value, return a `WrongArgumentCount` error if `f` does not take exactly one parameter. You may use `substitute`.

```
pub fn compose(f: UserFn, g: UserFn) -> Result(UserFn, EvalError) {
  case f.params {
    _____________ -> Ok(_____________________________________________)
    params -> Error(WrongArgumentCount(want: _____________, got: 1))
  }
}

assert compose(
    UserFn(["x"], Binary(Mul, Var("x"), Var("x"))),
    UserFn(["x", "y"], Binary(Sub, Var("y"), Var("x"))),
  )
  == Ok(UserFn(
    ["x", "y"],
    Binary(Mul, Binary(Sub, Var("y"), Var("x")), Binary(Sub, Var("y"),
       Var("x"))),
  ))
assert compose(
    UserFn(["x"], Binary(Mul, Var("x"), N(2))),
    UserFn(["x"], Binary(Add, Var("x"), N(3))),
  )
  == Ok(UserFn(["x"], Binary(Mul, Binary(Add, Var("x"), N(3)), N(2))))
assert compose(UserFn(["x", "y"], Binary(Sub, Var("y"), Var("x"))),
   UserFn(["x"], Var("x")))
  == Error(WrongArgumentCount(want: 2, got: 1))


pub fn compose(f: UserFn, g: UserFn) -> Result(UserFn, EvalError) {
  case f.params {
    [x] -> Ok(UserFn(g.params, substitute(f.body, x, g.body)))
    params -> Error(WrongArgumentCount(want: list.length(params), got: 1))
  }
}
```